

Phase 4 Notes: Master Theorem for Decreasing Functions

1. Main Goal of Phase 4

In Phase 4, we learn a shortcut for solving **decreasing recurrence relations**.

Before this phase, you solved recurrences manually using substitution. For example:

$$T(n) = T(n - 1) + 1$$

and:

$$T(n) = 2T(n - 1) + 1$$

In Phase 4, we use a shortcut called the **Master Theorem for Decreasing Functions**.

The goal is simple:

Instead of solving every decreasing recurrence step by step, we identify the pattern and quickly find the Big O answer.

2. Current Progress Before Phase 4

Before starting Phase 4, you already learned three important things.

From Phase 1

You learned how to look at recursive code and write a recurrence relation.

Example:

```
void printNumber(int n) {  
    if (n > 0) {  
        cout << n << " ";  
        printNumber(n - 1);  
    }  
}
```

```
}  
}
```

The recurrence is:

$$T(n) = T(n - 1) + 1$$
$$T(0) = 1$$

Meaning:

Part	Meaning
$T(n)$	Time for input size n
$T(n - 1)$	Time for the smaller recursive call
$+1$	Constant work done now
$T(0) = 1$	Base case takes constant time

From Phase 2

You learned how to solve one-call decreasing recurrences manually.

Example:

$$T(n) = T(n - 1) + 1$$

Using substitution:

$$T(n) = T(n - 1) + 1$$
$$T(n) = T(n - 2) + 2$$
$$T(n) = T(n - 3) + 3$$
$$T(n) = T(n - k) + k$$

Use the base case:

$$n - k = 0$$
$$k = n$$

Final result:

$T(n) = T(0) + n$
 $T(n) = 1 + n$
 $T(n) = O(n)$

You also learned these patterns:

Recurrence	Final Time
$T(n) = T(n - 1) + 1$	$O(n)$
$T(n) = T(n - 1) + n$	$O(n^2)$
$T(n) = T(n - 1) + n^2$	$O(n^3)$

From Phase 3

You learned that multiple recursive calls can create a tree.

Example:

```

void algorithm2(int n) {
    if (n > 0) {
        cout << n << " ";
        algorithm2(n - 1);
        algorithm2(n - 1);
    }
}

```

The recurrence is:

$T(n) = 2T(n - 1) + 1$
 $T(0) = 1$

This creates a branching tree:

```

      n
     /
  n - 1  n - 1
   / \   /
  ... ... ...

```

The final time complexity is:

$$O(2^n)$$

The key lesson was:

One recursive call creates a chain.
Two recursive calls create a tree.
A tree that keeps doubling grows exponentially.

3. What Is a Decreasing Recurrence?

A recurrence is called **decreasing** when the input becomes smaller by subtraction.

Examples:

n becomes n - 1
n becomes n - 2
n becomes n - 3

So these are decreasing recurrences:

$T(n) = T(n - 1) + 1$
 $T(n) = T(n - 2) + n$
 $T(n) = 2T(n - 1) + 1$
 $T(n) = 3T(n - 2) + n^2$

They are called decreasing because the input goes down by subtracting a number.

4. What Phase 4 Is Not For

Phase 4 is **not** for division recurrences.

These are not Phase 4 recurrences:

$T(n) = T(n / 2) + 1$
 $T(n) = 2T(n / 2) + n$
 $T(n) = 3T(n / 4) + n^2$

Those belong to the division master theorem, which comes later.

Important difference:

Type	Input Change	Example
Decreasing recurrence	subtracts from n	$T(n - 1)$
Dividing recurrence	divides n	$T(n / 2)$

Phase 4 is only for this kind:

$$T(n - b)$$

not this kind:

$$T(n / b)$$

5. General Form of the Master Theorem for Decreasing Functions

The general form is:

$$T(n) = aT(n - b) + f(n)$$

Each symbol has a meaning.

Symbol	Meaning
a	Number of recursive calls
b	Amount subtracted from n
$f(n)$	Extra work done outside the recursive calls

6. Meaning of a

The value a means:

How many recursive calls does the function make?

Example: One recursive call

```
function(n - 1);
```

There is one recursive call.

So:

```
a = 1
```

Example: Two recursive calls

```
function(n - 1);  
function(n - 1);
```

There are two recursive calls.

So:

```
a = 2
```

Example: Three recursive calls

```
function(n - 1);  
function(n - 1);  
function(n - 1);
```

There are three recursive calls.

So:

```
a = 3
```

7. Why a Is Very Important

For decreasing recurrences, a is the most important value.

Compare these two recurrences:

$$\begin{aligned}T(n) &= T(n - 1) + 1 \\T(n) &= 2T(n - 1) + 1\end{aligned}$$

They look similar, but their answers are very different.

Recurrence	Number of Calls	Shape	Time
$T(n) = T(n - 1) + 1$	1	Chain	$O(n)$
$T(n) = 2T(n - 1) + 1$	2	Tree	$O(2^n)$

The only big difference is a .

Memory idea:

$a = 1$ means chain.
 $a > 1$ means tree.

8. Meaning of b

The value b means:

How much is subtracted from n ?

Example 1

$$T(n) = T(n - 1) + 1$$

Here:

$$b = 1$$

because the input changes from n to $n - 1$.

Example 2

$$T(n) = T(n - 2) + 1$$

Here:

$$b = 2$$

because the input changes from n to $n - 2$.

Example 3

$$T(n) = 2T(n - 3) + n$$

Here:

$$b = 3$$

because the input changes from n to $n - 3$.

9. Meaning of $f(n)$

The value $f(n)$ means:

Extra work done outside the recursive calls.

This work may happen before the recursive call, after the recursive call, or around the recursive call.

Example 1: Constant work

```
cout << n;  
function(n - 1);
```

Printing once takes constant time.

So:

$$f(n) = 1$$

The recurrence is:

$$T(n) = T(n - 1) + 1$$

Example 2: Linear work

```
for (int i = 1; i <= n; i++) {  
    cout << i;  
}  
function(n - 1);
```

The loop runs `n` times.

So:

$$f(n) = n$$

The recurrence is:

$$T(n) = T(n - 1) + n$$

Example 3: Quadratic work

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        cout << "*";  
    }  
}  
function(n - 1);
```

The nested loops run about `n × n` times.

So:

$$f(n) = n^2$$

The recurrence is:

$$T(n) = T(n - 1) + n^2$$

10. The Three Cases in Phase 4

The decreasing master theorem has three cases:

Case 1: $a = 1$

Case 2: $a > 1$

Case 3: $a < 1$

The case depends mainly on the value of a .

Case 1: $a = 1$

11. Meaning of Case 1

Case 1 happens when the function makes exactly one recursive call.

The recurrence looks like this:

$$T(n) = T(n - b) + f(n)$$

Here:

$$a = 1$$

The recursive calls form a chain.

Example chain:

$$n \rightarrow n - 1 \rightarrow n - 2 \rightarrow n - 3 \rightarrow \dots \rightarrow 0$$

or:

$n \rightarrow n - 2 \rightarrow n - 4 \rightarrow n - 6 \rightarrow \dots$

The function does not branch. It only goes down one path.

12. Shortcut for Case 1

When:

$a = 1$

A simple beginner shortcut is:

$T(n) = O(n \times f(n))$

This means:

There are about n levels, and each level does $f(n)$ work.

This is a simplified rule for the patterns in this phase.

13. Case 1 Example: Constant Work

Recurrence:

$T(n) = T(n - 1) + 1$

Identify the parts:

$a = 1$
 $b = 1$
 $f(n) = 1$

Since $a = 1$, use Case 1:

$$O(n \times f(n))$$

Substitute $f(n) = 1$:

$$O(n \times 1) = O(n)$$

Final answer:

$$T(n) = O(n)$$

Simple meaning:

The function makes a chain of n calls.
Each call does constant work.
So total work is $O(n)$.

14. Case 1 Example: Linear Work

Recurrence:

$$T(n) = T(n - 1) + n$$

Identify the parts:

$$\begin{aligned} a &= 1 \\ b &= 1 \\ f(n) &= n \end{aligned}$$

Since $a = 1$, use Case 1:

$$O(n \times f(n))$$

Substitute $f(n) = n$:

$$O(n \times n) = O(n^2)$$

Final answer:

$$T(n) = O(n^2)$$

Simple meaning:

There are many calls.
Each call may do a loop.
The total work becomes quadratic.

15. Case 1 Example: Quadratic Work

Recurrence:

$$T(n) = T(n - 1) + n^2$$

Identify the parts:

$$\begin{aligned}a &= 1 \\b &= 1 \\f(n) &= n^2\end{aligned}$$

Since $a = 1$, use Case 1:

$$O(n \times f(n))$$

Substitute $f(n) = n^2$:

$$O(n \times n^2) = O(n^3)$$

Final answer:

$$T(n) = O(n^3)$$

Simple meaning:

The function walks down one chain.
At each step, it does square work.
Many square-work levels create cubic time.

16. Case 1 Example: Subtracting 2

Recurrence:

$$T(n) = T(n - 2) + 1$$

Identify the parts:

$$\begin{aligned}a &= 1 \\b &= 2 \\f(n) &= 1\end{aligned}$$

The calls go like this:

$n \rightarrow n - 2 \rightarrow n - 4 \rightarrow n - 6 \rightarrow \dots$

This gives about $n / 2$ levels.

But Big O ignores constant factors.

So:

$$O(n / 2) = O(n)$$

Final answer:

$$T(n) = O(n)$$

Important lesson:

For $a = 1$, changing b from 1 to 2 often changes only a constant factor.
Big O ignores constant factors.

17. Case 1 Pattern Table

Recurrence	a	b	f(n)	Final Time
$T(n) = T(n - 1) + 1$	1	1	1	$O(n)$
$T(n) = T(n - 1) + n$	1	1	n	$O(n^2)$
$T(n) = T(n - 1) + n^2$	1	1	n^2	$O(n^3)$
$T(n) = T(n - 2) + 1$	1	2	1	$O(n)$
$T(n) = T(n - 1) + \log n$	1	1	$\log n$	$O(n \log n)$

Case 2: $a > 1$

18. Meaning of Case 2

Case 2 happens when the function makes more than one recursive call.

The recurrence looks like this:

$$T(n) = aT(n - b) + f(n)$$

where:

$$a > 1$$

Examples:

$$\begin{aligned}T(n) &= 2T(n - 1) + 1 \\T(n) &= 3T(n - 1) + 1 \\T(n) &= 2T(n - 2) + n\end{aligned}$$

In this case, the recursive calls create a tree.

19. Why Case 2 Grows Fast

If there are two recursive calls, each call creates two more calls.

The number of calls grows like this:

1, 2, 4, 8, 16, 32, ...

This is exponential growth.

That is why decreasing recurrences with $a > 1$ usually become exponential.

Simple idea:

One call = chain.
More than one call = tree.
A tree that keeps branching grows very fast.

20. Shortcut for Case 2

When:

$a > 1$

The shortcut used in this phase is:

$T(n) = O(a^{(n / b)} \times f(n))$

Meaning:

Part	Meaning
a	Number of recursive calls
n / b	About how many levels until the base case
$f(n)$	Extra work done at each call

21. Case 2 Example: Two Calls, Constant Work

Recurrence:

$$T(n) = 2T(n - 1) + 1$$

Identify the parts:

$$\begin{aligned}a &= 2 \\b &= 1 \\f(n) &= 1\end{aligned}$$

Since $a > 1$, use Case 2:

$$O(a^{(n / b)} \times f(n))$$

Substitute the values:

$$O(2^{(n / 1)} \times 1)$$

Simplify:

$$O(2^n)$$

Final answer:

$$T(n) = O(2^n)$$

Simple meaning:

The function creates two smaller calls at every level.
The number of calls doubles again and again.
So the time is exponential.

22. Case 2 Example: Three Calls, Constant Work

Recurrence:

$$T(n) = 3T(n - 1) + 1$$

Identify the parts:

$a = 3$
 $b = 1$
 $f(n) = 1$

Since $a > 1$, use Case 2:

$O(a^{(n / b)} \times f(n))$

Substitute the values:

$O(3^{(n / 1)} \times 1)$

Simplify:

$O(3^n)$

Final answer:

$T(n) = O(3^n)$

Simple meaning:

Each call creates three more calls.
The tree grows even faster than the two-call version.

23. Case 2 Example: Two Calls, Subtracting 2

Recurrence:

$T(n) = 2T(n - 2) + 1$

Identify the parts:

$$\begin{aligned}a &= 2 \\ b &= 2 \\ f(n) &= 1\end{aligned}$$

Since $a > 1$, use Case 2:

$$O(a^{(n / b)} \times f(n))$$

Substitute the values:

$$O(2^{(n / 2)} \times 1)$$

Simplify:

$$O(2^{(n / 2)})$$

Final answer:

$$T(n) = O(2^{(n / 2)})$$

Important lesson:

For $a > 1$, b matters inside the exponent.

That means subtracting 2 instead of subtracting 1 can affect the exponential expression.

24. Case 2 Example: Two Calls, Linear Work

Recurrence:

$$T(n) = 2T(n - 1) + n$$

Identify the parts:

$a = 2$
 $b = 1$
 $f(n) = n$

Since $a > 1$, use Case 2:

$$O(a^{(n/b)} \times f(n))$$

Substitute the values:

$$O(2^{(n/1)} \times n)$$

Simplify:

$$O(n \times 2^n)$$

Final answer:

$$T(n) = O(n2^n)$$

or:

$$T(n) = O(n \times 2^n)$$

Simple meaning:

The branching already creates exponential growth.
The extra linear work adds an n factor.

25. Case 2 Pattern Table

Recurrence	a	b	f(n)	Final Time
$T(n) = 2T(n - 1) + 1$	2	1	1	$O(2^n)$
$T(n) = 3T(n - 1) + 1$	3	1	1	$O(3^n)$

Recurrence	a	b	f(n)	Final Time
$T(n) = 2T(n - 2) + 1$	2	2	1	$O(2^{(n/2)})$
$T(n) = 2T(n - 1) + n$	2	1	n	$O(n \times 2^n)$

Case 3: $a < 1$

26. Meaning of Case 3

Case 3 happens when:

$$a < 1$$

Example:

$$T(n) = 0.5T(n - 1) + n$$

This is mostly theoretical.

In real programming, a function cannot call itself half a time.

You can have:

1 recursive call
2 recursive calls
3 recursive calls

But you cannot normally have:

0.5 recursive calls

So this case is mainly a mathematical idea.

27. Shortcut for Case 3

When:

$$a < 1$$

The recursive part becomes less important.

The extra work dominates.

So:

$$T(n) = O(f(n))$$

Example:

$$T(n) = 0.5T(n - 1) + n$$

Here:

$$f(n) = n$$

So:

$$T(n) = O(n)$$

Again, this is not common in normal recursive code.

28. Case 3 Main Idea

Remember:

`a < 1` is theoretical.
It is not common in real recursive programs.
If it appears in math, the answer is usually `O(f(n))`.

29. Code Patterns for Phase 4

Pattern 1: One Call, Constant Work

```
void oneCallConstantWork(int n) {  
    if (n > 0) {  
        cout << n << " ";  
        oneCallConstantWork(n - 1);  
    }  
}
```

Breakdown:

One recursive call: $a = 1$
Input decreases by 1: $b = 1$
Prints once: $f(n) = 1$

Recurrence:

$$T(n) = T(n - 1) + 1$$

Final time:

$$O(n)$$

Pattern 2: Two Calls, Constant Work

```
void twoCallsConstantWork(int n) {  
    if (n > 0) {  
        cout << n << " ";  
        twoCallsConstantWork(n - 1);  
        twoCallsConstantWork(n - 1);  
    }  
}
```

Breakdown:

Two recursive calls: $a = 2$
Input decreases by 1: $b = 1$
Prints once: $f(n) = 1$

Recurrence:

$$T(n) = 2T(n - 1) + 1$$

Final time:

$$O(2^n)$$

Pattern 3: One Call, Linear Work

```
void oneCallLinearWork(int n) {  
    if (n > 0) {  
        for (int i = 1; i <= n; i++) {  
            cout << i << " ";  
        }  
        oneCallLinearWork(n - 1);  
    }  
}
```

Breakdown:

One recursive call: $a = 1$
Input decreases by 1: $b = 1$
Loop runs n times: $f(n) = n$

Recurrence:

$$T(n) = T(n - 1) + n$$

Final time:

$$O(n^2)$$

Pattern 4: Two Calls, Linear Work

```
void twoCallsLinearWork(int n) {  
    if (n > 0) {  
        for (int i = 1; i <= n; i++) {  
            cout << i << " ";  
        }  
        twoCallsLinearWork(n - 1);  
        twoCallsLinearWork(n - 1);  
    }  
}
```

Breakdown:

Two recursive calls: $a = 2$
Input decreases by 1: $b = 1$
Loop runs n times: $f(n) = n$

Recurrence:

$$T(n) = 2T(n - 1) + n$$

Final time:

$$O(n \times 2^n)$$

30. Dry Run Idea: Chain vs Tree

One-call decreasing recursion

Example:

$$T(n) = T(n - 1) + 1$$

Call shape for `n = 4`:

4 -> 3 -> 2 -> 1 -> 0

This is a chain.

There is only one path.

So the growth is slower.

Final time:

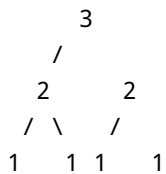
$O(n)$

Two-call decreasing recursion

Example:

$$T(n) = 2T(n - 1) + 1$$

Call shape for $n = 3$:



This is a tree.

The number of calls is:

$$1 + 2 + 4 = 7$$

For bigger n , it becomes:

$$1 + 2 + 4 + 8 + \dots$$

Final time:

$$O(2^n)$$

31. Step-by-Step Checklist for Solving Phase 4 Problems

When you see a recurrence, follow these steps.

Step 1: Check if it is decreasing

Look for:

$$\begin{aligned}T(n - 1) \\T(n - 2) \\T(n - 3)\end{aligned}$$

If you see this, it is decreasing.

If you see:

$$T(n / 2)$$

then it is not Phase 4.

Step 2: Match the general form

Write it in this form:

$$T(n) = aT(n - b) + f(n)$$

Step 3: Find **a**

Count the recursive calls.

Examples:

Code / Recurrence	a
$T(n) = T(n - 1) + 1$	1
$T(n) = 2T(n - 1) + 1$	2
$T(n) = 3T(n - 1) + 1$	3

Step 4: Find b

Look at what is subtracted from n.

Examples:

Recursive Part	b
$T(n - 1)$	1
$T(n - 2)$	2
$T(n - 3)$	3

Step 5: Find f(n)

Look at the extra work outside recursion.

Examples:

Extra Work	f(n)
print once	1
one loop from 1 to n	n
nested loops of size n × n	n ²
loop of size log n	log n

Step 6: Choose the correct case

Use a to choose the case.

Value of a	Case	Main Result
$a = 1$	Case 1	$O(n \times f(n))$
$a > 1$	Case 2	$O(a^{(n/b)} \times f(n))$
$a < 1$	Case 3	$O(f(n))$

32. Full Worked Examples

Example 1

Solve:

$$T(n) = T(n - 1) + 1$$

Step 1: Identify the form

$$T(n) = aT(n - b) + f(n)$$

Step 2: Find a , b , and $f(n)$

$$\begin{aligned} a &= 1 \\ b &= 1 \\ f(n) &= 1 \end{aligned}$$

Step 3: Choose the case

Since:

$$a = 1$$

Use Case 1.

Step 4: Apply the shortcut

$$O(n \times f(n))$$

Substitute:

$$O(n \times 1)$$

Final answer:

$$O(n)$$

Example 2

Solve:

$$T(n) = T(n - 1) + n$$

Step 1: Find a , b , and $f(n)$

$$\begin{aligned}a &= 1 \\ b &= 1 \\ f(n) &= n\end{aligned}$$

Step 2: Choose the case

Since:

$$a = 1$$

Use Case 1.

Step 3: Apply the shortcut

$$O(n \times f(n))$$

Substitute:

$$O(n \times n)$$

Final answer:

$$O(n^2)$$

Example 3

Solve:

$$T(n) = T(n - 1) + n^2$$

Step 1: Find a , b , and $f(n)$

$$\begin{aligned}a &= 1 \\b &= 1 \\f(n) &= n^2\end{aligned}$$

Step 2: Choose the case

Since:

$$a = 1$$

Use Case 1.

Step 3: Apply the shortcut

$$O(n \times f(n))$$

Substitute:

$$O(n \times n^2)$$

Final answer:

$$O(n^3)$$

Example 4

Solve:

$$T(n) = 2T(n - 1) + 1$$

Step 1: Find a , b , and $f(n)$

$$\begin{aligned}a &= 2 \\b &= 1 \\f(n) &= 1\end{aligned}$$

Step 2: Choose the case

Since:

$$a > 1$$

Use Case 2.

Step 3: Apply the shortcut

$$O(a^{(n / b)} \times f(n))$$

Substitute:

$$O(2^{(n / 1)} \times 1)$$

Simplify:

$$O(2^n)$$

Final answer:

$$O(2^n)$$

Example 5

Solve:

$$T(n) = 3T(n - 1) + 1$$

Step 1: Find a , b , and $f(n)$

$$\begin{aligned}a &= 3 \\b &= 1 \\f(n) &= 1\end{aligned}$$

Step 2: Choose the case

Since:

$$a > 1$$

Use Case 2.

Step 3: Apply the shortcut

$$O(a^{(n / b)} \times f(n))$$

Substitute:

$$O(3^{(n / 1)} \times 1)$$

Simplify:

$$O(3^n)$$

Final answer:

$$O(3^n)$$

Example 6

Solve:

$$T(n) = 2T(n - 2) + 1$$

Step 1: Find a , b , and $f(n)$

$$\begin{aligned}a &= 2 \\b &= 2 \\f(n) &= 1\end{aligned}$$

Step 2: Choose the case

Since:

$$a > 1$$

Use Case 2.

Step 3: Apply the shortcut

$$O(a^{(n / b)} \times f(n))$$

Substitute:

$$O(2^{(n / 2)} \times 1)$$

Final answer:

$$O(2^{(n / 2)})$$

Example 7

Solve:

$$T(n) = 2T(n - 1) + n$$

Step 1: Find a , b , and $f(n)$

$$\begin{aligned}a &= 2 \\b &= 1 \\f(n) &= n\end{aligned}$$

Step 2: Choose the case

Since:

$$a > 1$$

Use Case 2.

Step 3: Apply the shortcut

$$O(a^{(n / b)} \times f(n))$$

Substitute:

$$O(2^{(n / 1)} \times n)$$

Simplify:

$$O(n \times 2^n)$$

Final answer:

$$O(n \times 2^n)$$

33. Complete Program Example

This program shows one-call and two-call decreasing recursion.

```

#include <iostream>
using namespace std;

void oneCallConstantWork(int n) {
    if (n > 0) {
        cout << n << " ";
        oneCallConstantWork(n - 1);
    }
}

void twoCallsConstantWork(int n) {
    if (n > 0) {
        cout << n << " ";
        twoCallsConstantWork(n - 1);
        twoCallsConstantWork(n - 1);
    }
}

int main() {
    cout << "One call, n = 4: ";
    oneCallConstantWork(4);

    cout << "\nTwo calls, n = 3: ";
    twoCallsConstantWork(3);

    cout << endl;
    return 0;
}

```

Explanation of the Program

Function 1

```
oneCallConstantWork(n - 1);
```

There is one recursive call.

So:

```
a = 1
```

The recurrence is:

$$T(n) = T(n - 1) + 1$$

The final time is:

$$O(n)$$

Function 2

```
twoCallsConstantWork(n - 1);  
twoCallsConstantWork(n - 1);
```

There are two recursive calls.

So:

$$a = 2$$

The recurrence is:

$$T(n) = 2T(n - 1) + 1$$

The final time is:

$$O(2^n)$$

34. Time Complexity vs Stack Space

One-call decreasing recursion

Example:

$$T(n) = T(n - 1) + 1$$

Time:

$O(n)$

Stack space:

$O(n)$

Why?

The function creates a chain of calls:

$n \rightarrow n - 1 \rightarrow n - 2 \rightarrow \dots \rightarrow 0$

The deepest chain has about calls.

Two-call decreasing recursion

Example:

$T(n) = 2T(n - 1) + 1$

Time:

$O(2^n)$

Stack space:

$O(n)$

This may feel surprising.

The total number of calls is exponential, but the computer does not keep the whole tree on the stack at the same time.

It only keeps one active path at a time:

$n \rightarrow n - 1 \rightarrow n - 2 \rightarrow \dots \rightarrow 0$

So the maximum stack depth is still:

$O(n)$

35. Common Beginner Mistakes

Mistake 1: Using the wrong master theorem

Wrong idea:

Use Phase 4 for $T(n / 2)$

Correct idea:

Phase 4 is only for $T(n - b)$

Examples for Phase 4:

$T(n - 1)$
 $T(n - 2)$
 $T(n - 3)$

Not Phase 4:

$T(n / 2)$
 $T(n / 3)$

Mistake 2: Ignoring a

Some students do not count recursive calls carefully.

Compare:

$T(n) = T(n - 1) + 1$

and:

$$T(n) = 2T(n - 1) + 1$$

The first has:

$$a = 1$$

The second has:

$$a = 2$$

That changes the answer from:

$$O(n)$$

to:

$$O(2^n)$$

Mistake 3: Thinking two calls only doubles the time

A beginner may think:

Two recursive calls means the function is just 2 times slower.

But that is not correct.

The calls double at every level:

1, 2, 4, 8, 16, ...

So the time becomes exponential.

Mistake 4: Thinking b never matters

For $a = 1$, changing b often only changes a constant factor.

Example:

$$T(n) = T(n - 2) + 1$$

This is still:

$$O(n)$$

But for $a > 1$, b appears in the exponent.

Example:

$$T(n) = 2T(n - 2) + 1$$

This gives:

$$O(2^{(n / 2)})$$

So b can matter a lot in exponential cases.

Mistake 5: Treating $a < 1$ like normal code

In normal programming, recursive calls are counted as whole calls.

You can have:

```
1 call  
2 calls  
3 calls
```

But not:

```
0.5 calls
```

So $a < 1$ is mostly theoretical.

Mistake 6: Forgetting $f(n)$

The extra work matters.

Compare:

$$T(n) = 2T(n - 1) + 1$$

and:

$$T(n) = 2T(n - 1) + n$$

The first has:

$$f(n) = 1$$

The second has:

$$f(n) = n$$

So the answers are:

$$O(2^n)$$

and:

$$O(n \times 2^n)$$

36. Phase 4 Master Table

Case	Condition	Meaning	Shortcut	Example	Answer
Case 1	$a = 1$	One recursive call	$O(n \times f(n))$	$T(n)=T(n-1)+1$	$O(n)$

Case	Condition	Meaning	Shortcut	Example	Answer
Case 2	$a > 1$	Multiple recursive calls	$O(a^{(n/b)} \times f(n))$	$T(n)=2T(n-1)+1$	$O(2^n)$
Case 3	$a < 1$	Theoretical case	$O(f(n))$	$T(n)=0.5T(n-1)+n$	$O(n)$

37. Quick Practice Questions

Practice 1

$$T(n) = T(n - 1) + 1$$

Answer:

$$O(n)$$

Reason:

$$a = 1, f(n) = 1$$

$$O(n \times 1) = O(n)$$

Practice 2

$$T(n) = T(n - 1) + n$$

Answer:

$$O(n^2)$$

Reason:

$$a = 1, f(n) = n$$

$$O(n \times n) = O(n^2)$$

Practice 3

$$T(n) = T(n - 1) + n^2$$

Answer:

$$O(n^3)$$

Reason:

$$a = 1, f(n) = n^2$$
$$O(n \times n^2) = O(n^3)$$

Practice 4

$$T(n) = 2T(n - 1) + 1$$

Answer:

$$O(2^n)$$

Reason:

$$a = 2, b = 1, f(n) = 1$$
$$O(2^{(n/1)} \times 1) = O(2^n)$$

Practice 5

$$T(n) = 3T(n - 1) + 1$$

Answer:

$$O(3^n)$$

Reason:

$$a = 3, b = 1, f(n) = 1$$
$$O(3^{(n/1)} \times 1) = O(3^n)$$

Practice 6

$$T(n) = 2T(n - 2) + 1$$

Answer:

$$O(2^{(n/2)})$$

Reason:

$$a = 2, b = 2, f(n) = 1$$
$$O(2^{(n/2)} \times 1) = O(2^{(n/2)})$$

Practice 7

$$T(n) = 2T(n - 1) + n$$

Answer:

$$O(n \times 2^n)$$

Reason:

$$a = 2, b = 1, f(n) = n$$
$$O(2^{(n/1)} \times n) = O(n \times 2^n)$$

38. Simple Way to Remember Phase 4

Use this memory trick:

One call = chain.
More than one call = tree.

A chain grows slowly compared to a tree.

So:

$a = 1 \rightarrow \text{chain} \rightarrow \text{usually polynomial}$
 $a > 1 \rightarrow \text{tree} \rightarrow \text{exponential}$

Another memory trick:

$T(n - b)$ means decreasing recurrence.
 $T(n / b)$ means dividing recurrence.

Phase 4 is only for:

$T(n - b)$

39. Final Phase 4 Summary

Phase 4 teaches the master theorem shortcut for decreasing functions.

The general form is:

$T(n) = aT(n - b) + f(n)$

Where:

a = number of recursive calls
b = amount subtracted from n
f(n) = extra work outside recursion

There are three cases:

Case 1: $a = 1$
Case 2: $a > 1$
Case 3: $a < 1$

Case 1:

$a = 1$
 $T(n) = O(n \times f(n))$

Example:

$T(n) = T(n - 1) + 1 = O(n)$

Case 2:

$a > 1$
 $T(n) = O(a^{(n/b)} \times f(n))$

Example:

$T(n) = 2T(n - 1) + 1 = O(2^n)$

Case 3:

$a < 1$
 $T(n) = O(f(n))$

This case is theoretical and not common in real code.

The most important lesson is:

Count the recursive calls first.

The value of a decides whether the recursion is a chain or a tree.

40. What You Should Be Able to Do After Phase 4

After Phase 4, you should be able to:

1. Recognize a decreasing recurrence.
2. Know that decreasing means $T(n - b)$.
3. Avoid confusing $T(n - b)$ with $T(n / b)$.
4. Write the general form $T(n) = aT(n - b) + f(n)$.
5. Identify a , the number of recursive calls.
6. Identify b , the amount subtracted from n .
7. Identify $f(n)$, the extra work outside recursion.
8. Use Case 1 when $a = 1$.
9. Use Case 2 when $a > 1$.
10. Understand that $a < 1$ is theoretical.
11. Solve common decreasing recurrences quickly.
12. Explain why one call creates a chain.
13. Explain why multiple calls create a tree.
14. Explain why multiple decreasing calls often create exponential time.
15. Avoid common mistakes like using the division master theorem too early.

41. Key Formulas From Phase 4

General form:

$$T(n) = aT(n - b) + f(n)$$

Case 1:

$$a = 1$$
$$T(n) = O(n \times f(n))$$

Case 2:

$$a > 1$$
$$T(n) = O(a^{(n / b)} \times f(n))$$

Case 3:

$$a < 1$$
$$T(n) = O(f(n))$$

Most important memory trick:

One call = chain.
More than one call = tree.

Final reminder:

Use this phase only when the recurrence uses subtraction, like $T(n - 1)$ or $T(n - 2)$.
Do not use this phase for division recurrences like $T(n / 2)$.